



## Guía Didáctica — Proyecto Intermodular

Sergi Garcia Barea

CC BY-SA 4.0

# Guía Didáctica — Proyecto Intermodular

## Continuidad PI1 → PI2

 *El día que el proyecto volvió a empezar*

Septiembre. Segundo curso. El profesor dice: “El proyecto de PI1 era la base. Ahora, en PI2, lo vais a convertir en algo serio”.

Abres el código que escribiste hace 3 meses y no entiendes nada. Las variables se llaman `a` , `b` , `tmp` . Hay funciones de 300 líneas. Y un comentario que pone “no tocar o explota”.

Bienvenido a PI2. El proyecto no empieza de cero, pero a veces parece que sí.

## ¿Qué cambia de PI1 a PI2?

| Aspecto     | PI1              | PI2                             |
|-------------|------------------|---------------------------------|
| Enfoque     | Construir un MVP | Evolucionar a producto completo |
| Código base | Empezar de cero  | Partir del código de PI1        |
| Complejidad | Básico           | Profesional                     |
| Evaluación  | Retos            | Sprints                         |
| Entregas    | 5 retos          | 7 sprints                       |

## Cómo preparar la transición

### Durante los últimos meses de PI1

Grabáis el código de PI1 en un repositorio. Aseguraos de que el README explique cómo arrancar el proyecto y qué hace cada parte. No dejéis la documentación para el final.

## Primeras semanas de PI2

No empecéis a codificar inmediatamente. Primero:

1. **Revisad el código de PI1:** identificad qué salvar, qué rehacer, qué tirar
2. **Haced inventario de deuda técnica:** funciones sin test, código duplicado...
3. **Planificad la refactorización:** qué mejorar antes de añadir nada nuevo
4. **Decidid qué conservar:** no todo el código de PI1 merece sobrevivir

## Qué conservar de PI1

- La base de datos (si está bien diseñada)
- La lógica de negocio que funcione
- Las funcionalidades core (login, CRUD principal)
- La documentación que ya tengáis

## Qué rehacer en PI2

- La interfaz (si usabais HTML plano, ahora toca framework)
- El código sin tests
- Las partes que nunca llegaron a funcionar del todo
- La arquitectura (si no estaba clara)

► ? ¿Y si nuestro PI1 fue un desastre y apenas tenemos código aprovechable?

## PI1 — Proyecto Intermodular I

 *La primera vez que te enfrentas a un proyecto real*

Bienvenido al Proyecto Intermodular I. Durante este curso vas a desarrollar tu primera aplicación web completa. No es un ejercicio de clase con solución conocida. Es un **proyecto real** donde tú decides qué hacer y cómo hacerlo.

Da igual que tu idea sea grande o pequeña. Lo que importa es que la ejecutes bien.  
Y para eso están estos materiales.

## ¿Qué es PI1?

PI1 es la primera parte del Proyecto Intermodular de DAW. Se desarrolla durante el primer curso y consta de **5 retos** que te guían desde la identificación de una necesidad hasta la presentación de un MVP.

### Los 5 retos

| Reto                                                                                                          | Qué harás                                 | Entregable              |
|---------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------------------------|
|  <b>R1: Identificación</b>   | Definir el problema y la idea de solución | Documento de definición |
|  <b>R2: Análisis</b>         | Analizar requisitos y planificar          | Documento de análisis   |
|  <b>R3: Diseño</b>         | Diseñar la arquitectura y la interfaz     | Documento de diseño     |
|  <b>R4: Implementación</b> | Construir el MVP                          | Código funcional        |
|  <b>R5: Presentación</b>   | Defender el proyecto                      | Presentación + memoria  |

### Evaluación

Cada reto se evalúa de forma independiente. La nota final es la media ponderada según el peso de cada reto (consultar guía de evaluación).

## Competencias PI1

### Competencias generales

- **CG1:** Capacidad de análisis y síntesis para identificar problemas y proponer soluciones

- **CG2:** Capacidad de organización y planificación del trabajo
- **CG3:** Comunicación oral y escrita en la presentación de resultados
- **CG4:** Trabajo en equipo y colaboración con otros compañeros
- **CG5:** Aprendizaje autónomo y búsqueda de información técnica

## Competencias específicas

- **CE1:** Identificar necesidades del usuario y definir requisitos
- **CE2:** Diseñar la arquitectura de una aplicación web
- **CE3:** Implementar una aplicación web funcional (MVP)
- **CE4:** Utilizar herramientas de control de versiones
- **CE5:** Documentar el proceso de desarrollo
- **CE6:** Presentar y defender el proyecto ante un tribunal

## Relación con las asignaturas

| Asignatura             | Competencias que aporta             |
|------------------------|-------------------------------------|
| Programación           | Lógica, estructuras de datos, POO   |
| Bases de Datos         | Diseño y consulta de bases de datos |
| Lenguajes de Marcas    | HTML, XML, documentación            |
| Entornos de Desarrollo | Git, testing, depuración            |
| Sistemas Informáticos  | Instalación y configuración         |

## Reto 1: Identificación

 *El primer paso: encontrar un problema que merezca la pena*

Elegir el proyecto es la decisión más importante de PI1. Un proyecto bien elegido te motiva durante meses. Un proyecto mal elegido te arrastra hasta el final.

## Objetivos del reto

- Identificar un problema real que pueda resolverse con una aplicación web
- Definir el alcance y los objetivos del proyecto
- Validar la viabilidad de la solución propuesta

## Entregables

### 1. Documento de definición del proyecto (2-3 páginas):

- Descripción del problema
- Usuarios objetivo
- Solución propuesta
- Objetivos SMART
- Alcance (lo que incluye y lo que no)

### 2. Presentación inicial (5 minutos):

- Explicar el problema
- Mostrar la solución propuesta
- Justificar por qué es viable

► ? ¿Qué pasa si el profesor rechaza mi idea?



## Reto 2: Análisis

 *Antes de construir, hay que entender*

Tienes una idea. Ahora toca entenderla bien. No es lo mismo “una app de recetas” que entender que el usuario quiere filtrar por ingredientes que tiene en casa, en menos de 3 clics, y que las recetas aparezcan ordenadas por tiempo de preparación.

El análisis es el puente entre la idea y la solución concreta.

## Objetivos del reto

- Definir los requisitos funcionales y no funcionales del sistema
- Identificar los actores del sistema
- Documentar los casos de uso principales

## Entregables

### 1. Documento de análisis (3-5 páginas):

- Descripción detallada del problema
- Requisitos funcionales (RF-1, RF-2, ...)
- Requisitos no funcionales (rendimiento, seguridad, usabilidad)
- Diagrama de casos de uso
- Identificación de actores

### 2. Lista priorizada de funcionalidades:

- MVP (imprescindible para la entrega)
- Deseable (si hay tiempo)
- Futuro (fuera de alcance)

► ? ¿Cuántos requisitos tengo que definir?

## Reto 3: Diseño

### Los planos antes de la obra

No construirías una casa sin planos. Pues tu aplicación tampoco deberías programarla sin diseño. El Reto 3 es el momento de decidir cómo va a ser tu proyecto por dentro y por fuera.

## Objetivos del reto

- Diseñar la arquitectura del sistema (componentes, módulos, capas)
- Diseñar la base de datos (entidades, relaciones)
- Crear prototipos de la interfaz de usuario

- Documentar las decisiones técnicas

## Entregables

### 1. Documento de diseño (5-8 páginas):

- Diagrama de arquitectura
- Diagrama entidad-relación (modelo de datos)
- Decisiones técnicas justificadas
- Prototipos de pantallas (wireframes)
- Diseño de la navegación

### 2. Prototipo interactivo (opcional pero recomendable):

- Maqueta en Figma o similar
- O al menos bocetos en papel escaneados

► ? ¿Puedo cambiar el diseño durante la implementación?

## ⚙️ Reto 4: Implementación MVP

 *El momento de escribir código*

Has definido, analizado, diseñado. Ahora toca construir. Este es el reto más largo (y el que más peso tiene en la nota). Y también el que más satisfacción da cuando ves tu idea funcionando en el navegador.

## Objetivos del reto

- Implementar las funcionalidades definidas en el alcance MVP
- Asegurar que la aplicación funciona correctamente
- Organizar el código de forma mantenible
- Utilizar control de versiones (Git)



### 1. Código fuente:

- Repositorio Git con commits regulares
- Código organizado (estructura de carpetas clara)
- Comentarios en las partes no obvias

### 2. Aplicación funcional:

- Las funcionalidades MVP deben funcionar
- Interfaz usable (aunque sea básica)
- Datos reales (no solo de prueba)

### 3. Pequeña guía de uso (opcional):

- Cómo arrancar el proyecto
- Funcionalidades implementadas

► ? ¿Qué hago si no llego a implementar todo el MVP?

## Reto 5: Presentación

 *El día que todo el trabajo se resume en 10 minutos*

Has programado durante meses. Has sudado, has maldecido el teclado, has celebrado cuando algo funcionó. Ahora toca el momento de la verdad: 10 minutos para demostrar que todo el esfuerzo ha merecido la pena.

## Objetivos del reto

- Presentar el proyecto de forma clara y estructurada
- Demostrar que comprendes las decisiones técnicas tomadas
- Defender tu trabajo ante las preguntas del tribunal

## Entregables

### 1. Memoria del proyecto (15-30 páginas):

- Introducción y contexto
- Análisis y diseño
- Implementación (capturas de pantalla)
- Pruebas realizadas
- Conclusiones y trabajo futuro

2. **Presentación oral** (10-15 minutos):

- Diapositivas limpias y profesionales
- Demo en vivo de la aplicación
- Preparación para preguntas

3. **Repositorio final:** código completo y documentación

## Consejos para la defensa

- **Ensayo:** la presentación debe estar medida al segundo
- **Plan B:** lleva la demo en local, no dependas de internet
- **Sé honesto:** si algo no funciona, dilo y explica por qué
- **Mira al tribunal:** no leas diapositivas, háblales
- **Prepárate preguntas:** piensa qué te pueden preguntar y ten respuesta

---

## Evaluación PI1

## Criterios de evaluación

Cada reto se evalúa con los siguientes criterios generales:

| Criterio                           | Peso orientativo |
|------------------------------------|------------------|
| Cumplimiento de objetivos del reto | 30%              |
| Calidad técnica del trabajo        | 25%              |
| Documentación y memoria            | 20%              |
| Presentación y defensa             | 15%              |
| Trabajo en equipo (si aplica)      | 10%              |

## Sistema de calificación

Cada reto se califica de 0 a 10. La nota final de PI1 es la media ponderada:

| Reto               | Peso orientativo |
|--------------------|------------------|
| R1: Identificación | 10%              |
| R2: Análisis       | 15%              |
| R3: Diseño         | 20%              |
| R4: Implementación | 40%              |
| R5: Presentación   | 15%              |

**Nota:** cada reto tiene un peso distinto en función de su complejidad. Los pesos anteriores son orientativos; el profesor concretará los porcentajes al inicio del curso.

## Criterios de calidad

Para obtener más de un 5 en cada reto, el trabajo debe:

- **R1:** Idea viable, objetivos claros, alcance definido
- **R2:** Requisitos completos, análisis coherente
- **R3:** Arquitectura clara, diseño completo, prototipos
- **R4:** MVP funcional, código organizado, interfaz usable
- **R5:** Memoria completa, presentación clara, defensa solvente

- Los retos se entregan en las fechas indicadas
- La **falta de entrega** de un reto supone no superar la evaluación continua
- Las faltas de ortografía en la memoria penalizan
- El código debe ser original (no copiado de internet sin atribución)

## **PI2 — Proyecto Intermodular II**

 *La segunda oportunidad de hacerlo mejor*

Bienvenido a PI2. Ya tienes experiencia: sabes lo que es plantificar mal, lo que es no llegar a tiempo, lo que es tener un código que da vergüenza enseñar.

Ahora sabes mejor lo que NO hacer. Y eso te da ventaja.

## ¿Qué es PI2?

PI2 es la continuación del Proyecto Intermodular. Partes del MVP de PI1 y lo conviertes en un **producto completo y profesional**. Se organiza en **7 sprints** que cubren desde la auditoría del código existente hasta la defensa final.

## Los 7 sprints

| Sprint                                                                                                      | Qué harás                | Enfoque         |
|-------------------------------------------------------------------------------------------------------------|--------------------------|-----------------|
|  <b>S0: Auditoría</b>    | Revisar el código de PI1 | Refactorización |
|  <b>S1: Arquitectura</b> | Mejorar la estructura    | Diseño          |
|  <b>S2: Frontend</b>     | Mejorar la interfaz      | Framework       |
|  <b>S3: Backend</b>      | Mejorar la lógica        | APIs            |
|  <b>S4: Testing</b>      | Probar todo              | Calidad         |
|  <b>S5: Despliegue</b>   | Publicar la app          | Producción      |
|  <b>S6: Defensa</b>      | Presentar el proyecto    | Comunicación    |

## Competencias generales

- **CG1:** Capacidad para evolucionar y mantener software existente
- **CG2:** Integración de tecnologías y servicios externos
- **CG3:** Automatización de procesos (testing, despliegue)
- **CG4:** Trabajo en equipo avanzado (roles, metodologías ágiles)
- **CG5:** Compromiso con la calidad y la seguridad

## Competencias específicas

- **CE1:** Refactorizar y mejorar código existente
- **CE2:** Integrar APIs y servicios externos
- **CE3:** Implementar pruebas automatizadas
- **CE4:** Desplegar aplicaciones en entornos cloud
- **CE5:** Aplicar medidas de seguridad básicas
- **CE6:** Configurar flujos CI/CD
- **CE7:** Documentar la evolución del proyecto

## Relación con las asignaturas de 2º

| Asignatura                         | Competencias que aporta            |
|------------------------------------|------------------------------------|
| Desarrollo Web en Entorno Cliente  | Frontend avanzado, frameworks      |
| Desarrollo Web en Entorno Servidor | APIs, seguridad, despliegue        |
| Despliegue de Aplicaciones Web     | CI/CD, cloud, contenedores         |
| Diseño de Interfaces               | UX, usabilidad, prototipado        |
| Empresa                            | Modelos de negocio, emprendimiento |

 *Abrir el código de PI1 y no morir en el intento*

El primer sprint de PI2 no es escribir código nuevo. Es **mirar el que ya tienes** y ser honesto sobre su estado.

## Objetivos

- Auditar el código y la arquitectura de PI1
- Identificar deuda técnica y problemas
- Definir un plan de refactorización y mejora

## Entregables

### 1. Informe de auditoría:

- Estado del código (qué funciona, qué no, qué falta)
- Problemas identificados (deuda técnica, bugs)
- Puntuación de calidad por módulo

### 2. Plan de mejora:

- Qué refactorizar (priorizado por impacto)
- Qué mantener tal cual
- Qué rehacer desde cero
- Estimación de tiempo para cada mejora

## Preguntas guía

- ¿El código está organizado en capas o es un monolito?
- ¿Las funciones hacen una sola cosa?
- ¿Los nombres de variables y funciones son descriptivos?
- ¿Hay tests? ¿Pasan?
- ¿La base de datos está normalizada?
- ¿La interfaz es responsive?
- ¿El proyecto se puede arrancar en otra máquina?



## Sprint 1: Arquitectura



### *Poner orden en la casa*

Con la auditoría hecha, sabes qué duele. Ahora toca arreglarlo. Este sprint es para reordenar la arquitectura antes de añadir nada nuevo.

## Objetivos

- Refactorizar la arquitectura según el plan del Sprint o
- Separar responsabilidades (capas, módulos)
- Aplicar patrones de diseño donde tenga sentido
- Documentar la nueva arquitectura

## Entregables

1. **Código refactorizado:** estructura de carpetas actualizada
2. **Diagrama de arquitectura:** versión mejorada
3. **Justificación de cambios:** por qué se ha cambiado cada cosa

## Consejos

- No intentes refactorizar todo a la vez. Prioriza los cambios que más impacto tienen en la mantenibilidad
- Cada refactorización debe ir acompañada de commits atómicos
- Si una parte funciona bien, no la toques solo por “mejorarla”



## Sprint 2: Frontend



### *Darle una vuelta a la interfaz*

En PI1 la interfaz cumplía. En PI2 tiene que **brillar**. Este sprint es para mejorar la experiencia de usuario y dar un salto de calidad visual.

## Objetivos

- Migrar o mejorar la interfaz usando un framework frontend
- Crear componentes reutilizables
- Mejorar la experiencia de usuario
- Asegurar que es responsive y accesible

## Entregables

1. **Código frontend mejorado**: componentes organizados
2. **Prototipos** de las nuevas pantallas
3. **Documentación** de los componentes creados

## Consejos

- No intentes aprender el framework perfecto. Aprende el justo para tu proyecto
- Los componentes atómicos (botón, input, tarjeta) son los que más reutilización aportan
- Prueba la interfaz con alguien que no sea del equipo

---

## Sprint 3: Backend y APIs

 *El backend se hace mayor*

En PI1 el backend era funcional pero básico. Ahora toca añadirle las capas de validación, seguridad y documentación que necesita un producto serio.

## Objetivos

- Mejorar la API con validación y manejo de errores



- Implementar autenticación y autorización (JWT)
- Documentar la API (Swagger/OpenAPI)
- Asegurar el backend contra vulnerabilidades comunes

## Entregables

1. **API mejorada:** endpoints con validación y errores consistentes
2. **Autenticación JWT:** registro, login, rutas protegidas
3. **Documentación de la API:** Swagger o similar
4. **Mejoras de seguridad:** consultas parametrizadas, hash de contraseñas

## Consejos

- Valida siempre en el servidor, nunca confíes en el frontend
- Los códigos HTTP correctos son importantes (201, 400, 401, 404)
- Documenta la API aunque solo la uses tú

---

## Sprint 4: Testing

 *Probar para no fallar*

Los tests no son un extra. Son la red de seguridad que te permite cambiar código sin miedo a romperlo todo. En PI2 son obligatorios.

## Objetivos

- Escribir tests unitarios para las funciones críticas
- Escribir tests de integración para los flujos principales
- Configurar ejecución automática de tests (CI)
- Alcanzar una cobertura mínima en las partes importantes

## Entregables

1. **Tests unitarios** (Jest, Vitest, JUnit)

2. **Tests de integración** de la API
3. **CI configurado** (GitHub Actions ejecutando tests)
4. **Informe de cobertura** (opcional)

## Consejos

- Empieza por lo que más puede romperse (login, CRUD principal)
- Escribe tests para bugs que ya hayas encontrado (regresión)
- Los tests deben ser rápidos (si tardan 5 minutos, no los ejecutarás)
- Un test que no se ejecuta automáticamente no sirve

## Sprint 5: Despliegue

 *Sacar la app al mundo*

Tu aplicación funciona en localhost. Pero el tribunal no va a venir a tu casa a verla. Tiene que estar accesible desde cualquier navegador con una URL.

## Objetivos

- Desplegar la aplicación en un servidor cloud
- Configurar CI/CD (despliegue automático desde Git)
- Asegurar HTTPS
- Documentar el proceso de despliegue

## Entregables

1. **URL pública** de la aplicación funcionando
2. **CI/CD configurado** (GitHub Actions u otro)
3. **Documentación del despliegue** (pasos, configuración)

## Consejos

- Prueba el despliegue varios días antes de la entrega

- Las plataformas gratuitas (Render, Vercel, Railway) son suficientes
- HTTPS es obligatorio si hay formularios de login
- La URL debe estar en la memoria del proyecto
- Si la app tarda en arrancar (plan gratuito), avísalo en la memoria

## Sprint 6: Defensa

 *El último sprint: demostrar todo lo aprendido*

El proyecto entero ha sido el camino. La defensa es la meta. Este sprint no es para programar, es para **comunicar** todo lo que has hecho.

## Objetivos

- Completar la memoria final del proyecto
- Preparar la presentación oral
- Ensayar la defensa
- Dejar el repositorio listo para la evaluación

## Entregables

1. **Memoria final** completa (incluye evolución desde PI1)
2. **Presentación oral** (diapositivas + demo)
3. **Repositorio finalizado** (README, licencia, tags)

## Estructura de la memoria final

1. Portada e índice
2. Introducción (contexto, objetivos)
3. Evolución desde PI1 (qué cambió y por qué)
4. Arquitectura final
5. Implementación destacada (código relevante)
6. Tests y calidad

7. Despliegue y CI/CD
8. Problemas encontrados y soluciones
9. Conclusiones y trabajo futuro
10. Bibliografía y referencias

## Estructura de la presentación (10-15 min)

1. El problema que resuelve (1 min)
2. Demo de la aplicación (3-4 min)
3. Arquitectura y decisiones técnicas (2-3 min)
4. Lo más difícil y cómo lo superaste (2 min)
5. Evolución desde PI1 (1 min)
6. Conclusiones (1 min)

## Consejos para la defensa

- **Ensayá 3 veces** mínimo. La primera para ver si te pasas de tiempo
  - **No leas**: habla natural, mira al tribunal
  - **Ten plan B**: demo grabada por si falla el directo
  - **Prepárate preguntas**: API, seguridad, decisiones técnicas, learnings
  - **Muestra entusiasmo**: si a ti te gusta tu proyecto, se nota
- 



## Entregables PI2

## Entregables por sprint

### Sprint 0: Auditoría

- Informe de auditoría del código de PI1
- Plan de mejora y refactorización

### Sprint 1: Arquitectura

- Diagrama de arquitectura mejorado

- Decisiones técnicas justificadas
- Estructura de carpetas actualizada

## **Sprint 2: Frontend**

- Interfaz mejorada (framework de componentes)
- Prototipos de las nuevas pantallas
- Código frontend organizado

## **Sprint 3: Backend**

- API mejorada con validación y seguridad
- Documentación de la API (Swagger o similar)
- Código backend organizado

## **Sprint 4: Testing**

- Tests unitarios de funciones críticas
- Tests de integración del flujo principal
- Informe de cobertura (opcional)

## **Sprint 5: Despliegue**

- Aplicación desplegada (URL accesible)
- CI/CD configurado (GitHub Actions)
- README con instrucciones actualizadas

## **Sprint 6: Defensa**

- Memoria final completa
- Presentación oral
- Repositorio finalizado

## **Formato de entrega**

Todos los entregables se suben al repositorio de GitHub en la rama correspondiente. La memoria y documentación pueden estar en Markdown dentro del propio repositorio o en PDF.

## Criterios de evaluación

| Criterio                 | Peso orientativo |
|--------------------------|------------------|
| Evolución respecto a PI1 | 20%              |
| Calidad técnica          | 25%              |
| Testing y calidad        | 15%              |
| Despliegue y CI/CD       | 15%              |
| Documentación y memoria  | 15%              |
| Presentación y defensa   | 10%              |

## Sistema de calificación

Cada sprint se califica de 0 a 10. La nota final de PI2 es la media ponderada:

| Sprint           | Peso orientativo |
|------------------|------------------|
| So: Auditoría    | 5%               |
| S1: Arquitectura | 10%              |
| S2: Frontend     | 15%              |
| S3: Backend      | 15%              |
| S4: Testing      | 15%              |
| S5: Despliegue   | 20%              |
| S6: Defensa      | 20%              |

**Nota:** cada sprint tiene un peso distinto en función de su complejidad. Los pesos anteriores son orientativos; el profesor concretará los porcentajes al inicio del curso.

Para obtener más de un 5 en cada sprint:

- **S0:** Auditoría honesta y plan de mejora realista
- **S1:** Arquitectura mejorada y justificada
- **S2:** Framework frontend con componentes
- **S3:** API mejorada con validación y seguridad
- **S4:** Tests automatizados en partes críticas
- **S5:** App desplegada y accesible por URL
- **S6:** Memoria completa y presentación profesional

## Observaciones

- La **evolución** respecto a PI1 se valora positivamente
- El despliegue en producción (aunque sea gratuito) suma puntos
- Los tests automatizados son obligatorios en PI2
- La memoria debe incluir la comparativa con PI1

## Metodología

 *El día que entendieron que la metodología no era burocracia*

“¿Otra reunión?”, “¿Otra columna en el tablero?”, “¿Otra estimación?”. Al principio, todo lo que sonaba a “metodología” les parecía pérdida de tiempo. Querían programar, no hacer reuniones.

Hasta que llegó la semana antes de la entrega y descubrieron: - Una funcionalidad que daban por hecha no estaba empezada - Dos personas habían trabajado en lo mismo sin saberlo - Nadie sabía qué quedaba por hacer exactamente

La metodología no es burocracia. Es **no llegar en pánico al final**.

## ¿Qué metodología usar?

No hay una única respuesta. Depende de tu equipo y tu proyecto.

Sprints de 1-2 semanas. Reuniones cortas con propósito. Roles rotativos. Ideal para fechas de entrega fijas.

### ✅ Kanban (recomendado para PI2)

Flujo continuo sin sprints. Límites de trabajo en curso (WIP). Menos reuniones. Ideal para mantenimiento y mejoras.

## SCRUM (adaptado)

Para equipos de 2-4 personas, con sprints de 1-2 semanas.

**Qué hacer:** - Sprint planning al empezar cada sprint (qué haremos, quién lo hará) - Daily (15 min, de pie o sentados, pero rápidos) - Sprint review al terminar (qué se ha conseguido) - Retrospectiva (qué mejorar para el próximo sprint)

**Qué NO hacer:** - Scrum Master oficial (rotad el rol) - Ceremonias de 2 horas - Estimar en puntos de historia (usad horas o días) - Sprint backlog inmutable (se puede renegociar)

## Kanban

Para equipos que prefieren fluidez continua sin sprints fijos.

**Qué hacer:** - Tablero con columnas: Pendiente → En curso → Revisión → Hecho - Límite de tareas en “En curso” (máximo 2 por persona) - Flujo continuo sin fechas fijas

**Ventaja:** más flexible, menos reuniones. **Desventaja:** menos estructura, fácil procrastinar.

## Git Flow

No es solo para el código. También organiza el trabajo:

| Rama      | Propósito                                 |
|-----------|-------------------------------------------|
| main      | Código estable y desplegado               |
| develop   | Integración de funcionalidades            |
| feature/* | Una funcionalidad nueva (una por persona) |
| hotfix/*  | Corrección urgente                        |



# El ciclo del proyecto

Sea cual sea la metodología, el proyecto sigue este ciclo:

\*\* 📅 Planificar →

→ → \*\* ⚡ Ejecutar →

→ → \*\* 🔍 Revisar →

→ → \*\* 💡 Aprender → →

Cada iteración (semana, sprint) deberías: 1. Elegir qué hacer 2. Hacerlo 3. Comprobar que funciona 4. Reflexionar sobre lo aprendido

Sin el paso 4, repites los mismos errores. Sin el paso 1, trabajas sin dirección.

## 📅 Caso 1: El equipo que no planificaba

Un equipo de 3 personas decidió que “la metodología era pérdida de tiempo”. Se pusieron a programar directamente desde el día 1.

**Resultado:** - A 3 días de la entrega, descubrieron que dos personas habían implementado la misma funcionalidad - Una feature crítica no se había empezado - Tuvieron que dormir 4 horas al día para llegar

**Lección:** 30 minutos de planificación al inicio de la semana habrían ahorrado 3 días de caos.

## Ejemplo de sprint de 2 semanas

### Semana 1

| Día       | Mañana                   | Tarde            |
|-----------|--------------------------|------------------|
| Lunes     | Sprint planning (30 min) | Desarrollo       |
| Martes    | Daily (15 min)           | Desarrollo       |
| Miércoles | Daily (15 min)           | Desarrollo       |
| Jueves    | Daily (15 min)           | Desarrollo       |
| Viernes   | Daily (15 min)           | Revisión parcial |

### Semana 2

| Día       | Mañana                         | Tarde           |
|-----------|--------------------------------|-----------------|
| Lunes     | Daily (15 min)                 | Desarrollo      |
| Martes    | Daily (15 min)                 | Desarrollo      |
| Miércoles | Daily (15 min)                 | Desarrollo      |
| Jueves    | Daily (15 min)                 | Últimas pruebas |
| Viernes   | Sprint review + Retro (45 min) | 🎨 Descanso      |

Total de reuniones: ~**3 horas en 2 semanas** (menos del 5% del tiempo).

## Errores comunes

- **Sobreplanificar:** perder 2 días planificando un sprint de 1 semana
- **Infraplanificar:** empezar a programar sin saber qué hace cada uno
- **No hacer retrospectiva:** repetir los mismos errores sprint tras sprint
- **Daily que se alarga:** las daily son de 15 min, no de 45

## Herramientas recomendadas

| Herramienta              | Para qué                  |
|--------------------------|---------------------------|
| Trello / GitHub Projects | Tablero Kanban            |
| Notion / Confluence      | Documentación compartida  |
| Discord / Slack          | Comunicación diaria       |
| Google Calendar          | Reuniones y fechas límite |
| Git + GitHub / GitLab    | Control de versiones      |

### *El día que se dieron cuenta de que no necesitaban 20 herramientas*

El equipo empezó con entusiasmo: un chat para hablar, otro para compartir archivos, un tablero para tareas, un documento para ideas, otro documento para la memoria, un repositorio, un gestor de contraseñas...

A las dos semanas, la información estaba dispersa en 8 sitios distintos. Nadie recordaba dónde estaba cada cosa. Perdían más tiempo buscando información que creándola.

Menos herramientas. Mejor organizadas.

## Herramientas esenciales

No necesitas más de 4-5 herramientas bien usadas.

### 1. Repositorio de código (Git + GitHub/GitLab)

Tu repositorio no es solo para el código. También es para: - Documentación técnica (README, Wiki) - Issues (tareas, bugs, ideas) - Pull requests (revisión de código) - Projects (tablero Kanban integrado)

**Tip:** usa GitHub Projects como tablero. Así tienes código y tareas en el mismo sitio. Una herramienta menos que gestionar.

### 2. Comunicación (Discord, WhatsApp, Slack)

Elegid una. Solo una. No combinéis WhatsApp para “urgente” y Discord para “lo demás”. Todo en el mismo sitio.

**Consejo:** cread canales o grupos separados para: - General (anuncios, dudas generales) - Técnico (problemas de código) - Off-topic (memes, no interrumpir a los demás)

### 3. Documentación (Markdown en el repo)

La memoria del proyecto, las decisiones técnicas, los diagramas. Todo en un sitio accesible para todos.

**Recomendación:** usad Markdown en el propio repositorio. Así la documentación viaja con el código y tenéis control de versiones.

### 4. Gestión de tareas (GitHub Projects, Trello, Notion)

Un tablero con tres columnas basta:

\*\* 📄 Por hacer →

→ → \*\* ⚙ En proceso →

→ → \*\* ✅ Hecho → →

Si necesitas más, añade “Revisión” o “Bloqueado”. Pero no te pases. Cuantas más columnas, más tiempo pierdes moviendo tarjetas.

### 📄 **Caso 2: El día que consolidaron sus herramientas**

Un equipo usaba: - WhatsApp para hablar - Trello para tareas - Google Docs para documentación - GitHub para código - Figma para diseños - Drive para assets

**El problema:** cada vez que necesitaban algo, no sabían dónde buscar. La información estaba fragmentada.

**La solución:** - Movieron todo a GitHub (Issues + Projects + Wiki) - Markdown para documentación (dentro del repo) - Discord para comunicación (integrado con GitHub) - Figma solo para prototipos iniciales

**Resultado:** 4 herramientas → 3. La información, centralizada.

## 5. IDE (VS Code recomendado)

VS Code es el estándar por su ecosistema de extensiones:

| Extensión            | Para qué                            |
|----------------------|-------------------------------------|
| ESLint / Prettier    | Código limpio y formateado          |
| GitLens              | Historial y blame en línea          |
| GitHub Pull Requests | Revisar PRs sin salir del IDE       |
| Live Share           | Programar en pareja en remoto       |
| Docker               | Gestionar contenedores              |
| Thunder Client       | Probar APIs (alternativa a Postman) |

## Herramientas complementarias

| Herramienta        | Para qué                        | Alternativas         |
|--------------------|---------------------------------|----------------------|
| Postman / Insomnia | Probar APIs                     | Hoppscotch           |
| Draw.io / Figma    | Diagramas y prototipos          | Excalidraw, Mermaid  |
| Docker             | Contenedores                    | Podman               |
| Vercel / Netlify   | Despliegue frontend             | Render, GitHub Pages |
| UptimeRobot        | Monitorizar que la app responde | —                    |

## Flujo de trabajo típico

1. Abres una Issue en GitHub → tarea pendiente
2. Mueves la Issue a "En proceso" → estás trabajando
3. Creas una rama feature/ desde develop → código
4. Abres un Pull Request → revisión del compañero
5. Mergeas a develop → integración
6. Cierras la Issue → 

## Lo que NO necesitas

- Un IDE distinto cada uno (cada uno que use el que quiera)
- Una herramienta de gestión de proyectos solo porque está de moda
- Un servicio cloud que nadie sabe configurar
- Una base de datos externa cuando SQLite local es suficiente
- Un canal de comunicación distinto para cada tema

## Recursos

 *El día que Stack Overflow era su mejor amigo sin Internet*

“No me funciona este código”, dijo una voz en el grupo. “Búscalo en Google”, respondió otra.

Buscaron. Encontraron una respuesta en Stack Overflow de 2015. La probaron. No funcionaba. Buscaron otra. Esa sí.

## Cómo buscar ayuda técnica

### Busca en el orden correcto

1. **Documentación oficial** (la fuente más fiable)
2. **Stack Overflow** (problemas comunes ya resueltos)
3. **GitHub Issues** (bugs conocidos, soluciones de la comunidad)
4. **Tutoriales / blogs** (guías paso a paso)
5. **Pregunta a un compañero o al profesor** (cuando lleves 30 min atascado)

### Cómo preguntar bien

Cuando preguntes (en Stack Overflow, al profesor, a un compañero), sé específico:

❌ “No me funciona el login”    ✅ “Al enviar el formulario de login con email y contraseña válidos, recibo un error 500 en la consola del servidor. El código relevante es...”

Incluye siempre: - Qué esperabas que pasara - Qué pasa realmente - El código relevante (no todo el archivo) - Los mensajes de error completos

#### **Caso 3: El día que aprendieron a buscar**

Un alumno llevaba 3 horas atascado con un error de conexión a base de datos. Había probado “cosas” sin saber bien qué hacía.

**Su proceso:** - Pegó el error en Google - Probó la primera respuesta (no funcionó) - Probó la segunda (tampoco) - Se rindió y preguntó al profesor

**El profesor hizo:** - Copió el error exacto en Google - Encontró un GitHub Issue con el mismo error - La solución era cambiar una línea de configuración

**Lección:** el error estaba documentado en GitHub Issues, pero él no sabía buscar allí. Aprender a buscar es tan importante como aprender a programar.

## Referencias por tema

- [MDN Web Docs](#) — la biblia del desarrollo web
- [CSS-Tricks](#) — guías visuales de CSS
- Documentación oficial de React / Vue / Svelte

## Backend

- [Express.js](#) guía oficial
- [Node.js](#) documentación de la API
- [Spring Boot](#) guía oficial

## Bases de datos

- Documentación oficial de PostgreSQL / MySQL
- [SQL Tutorial](#) (w3schools, para empezar)

## Git

- [Pro Git](#) — libro gratuito y completo
- [Oh Shit, Git!?!](#) — cuando algo sale mal

## Docker

- [Docker Curriculum](#) — tutorial interactivo
- Documentación oficial de Docker

## IA como recurso de aprendizaje

▶  ¿Puedo usar ChatGPT/Claude para el proyecto?

## El recurso más infravalorado: tu profesor

El profesor no es solo para evaluar. Es tu **recurso principal**. Pregúntale: - ¿Voy por buen camino? - ¿Esta decisión técnica tiene sentido? - ¿Cómo harías tú esto? - ¿Qué priorizarías ahora?

Una consulta de 5 minutos al profesor puede ahorrarte 2 días de trabajo.



## *Quién te guía en el proyecto*

El Proyecto Intermodular no es un viaje en solitario. Detrás hay un equipo docente que te acompaña, orienta y evalúa. Conócelos.

## Coordinador del proyecto

| Profesor           | Rol                                                   |
|--------------------|-------------------------------------------------------|
| Sergi Garcia Barea | Coordinador PI — Programación, Entornos de Desarrollo |

## ¿Qué hace cada profesor?

El proyecto es **intermodular**, lo que significa que involucra varias asignaturas. Cada profesor aporta desde su área:

- **Programación** — supervisa la calidad del código, la arquitectura y el cumplimiento de los requisitos técnicos
- **Bases de Datos** — valida el diseño de datos, las consultas y la integridad de la información
- **Lenguajes de Marcas** — revisa la documentación, la memoria y los formatos de entrega
- **Entornos de Desarrollo** — guía en el uso de Git, testing y herramientas de desarrollo
- **Diseño de Interfaces** — asesora en wireframes, prototipos y experiencia de usuario

## Cómo contactar

- **Horario de clase:** plantea tus dudas durante las sesiones
- **Tutorías:** consulta el horario disponible en el centro
- **GitHub:** abre un issue en el repositorio del proyecto para dudas técnicas que puedan quedar registradas



El profesor no está para resolverte el proyecto, está para que aprendas a resolverlo tú. Pregunta, equivócate, pregunta otra vez.

## Licencia

 *Compartir es aprender*

Todo el contenido de esta guía se publica con licencia abierta para que puedas usarlo, adaptarlo y compartirlo. El conocimiento no se guarda, se distribuye.

Esta guía didáctica y todos los materiales asociados (apuntes, plantillas, ejemplos) se publican bajo la licencia **Creative Commons Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0)**.

## Eres libre de

- **Compartir** — copiar y redistribuir el material en cualquier medio o formato
- **Adaptar** — remezclar, transformar y crear a partir del material

## Bajo las siguientes condiciones

- **Atribución** — debes reconocer la autoría de forma adecuada, proporcionar un enlace a la licencia e indicar si se han realizado cambios
- **CompartirIgual** — si remezclas, transformas o creas a partir del material, deberás difundir tus contribuciones bajo la **misma licencia** que el original

## Autoría

**Sergi Garcia Barea** — CC BY-SA 4.0

 CC BY-SA 4.0  
CC BY-SA 4.0

Más información en: <https://creativecommons.org/licenses/by-sa/4.0/>

## ? Preguntas Frecuentes

 *Las dudas que todo el mundo tiene*

Da igual el curso: siempre surgen las mismas preguntas. Aquí tienes las respuestas para que no pierdas tiempo ni te lleves sorpresas.

### Sobre el proyecto

► ? ¿Puedo cambiar de idea a mitad de proyecto?

► ? ¿Qué pasa si mi equipo se rompe?

► ? ¿Puedo hacer el proyecto individual?

► ? ¿Qué pasa si el profesor rechaza mi idea?

► ? ¿Puedo usar inteligencia artificial?

### Sobre la evaluación

► ? ¿Qué peso tiene cada reto/sprint?

► ? ¿Qué pasa si no entrego un reto a tiempo?

► ? ¿La presentación oral cuenta?

► ? ¿Se evalúa el trabajo en equipo?

## Sobre el código

► ? ¿Puedo usar código de internet?

► ? ¿Cuánto código tengo que escribir?

► ? ¿Tengo que hacer tests?

► ? ¿Qué pasa si mi código no compila?

## Sobre la documentación

► ? ¿La memoria se entrega al final?

► ? ¿Qué extensión debe tener?

► ? ¿Tengo que hacer una memoria en PDF?

► ? ¿Tienes otra duda?



## Ideas para Proyectos

 *El día que no se les ocurría nada*

“Vale, tenéis que hacer un proyecto. Ideas, ideas...” Silencio en la clase. Nadie sabía qué hacer.

No hace falta una idea revolucionaria. Hace falta una idea que **resuelva un problema real**, aunque sea pequeño.

# Cómo elegir una buena idea

|            | Buena idea                    | Mala idea                            |
|------------|-------------------------------|--------------------------------------|
| Alcance    | Resuelve un problema concreto | “Una red social para todo”           |
| Usuario    | Tiene un usuario definido     | “Es para todo el mundo”              |
| Tecnología | Usa lo que sabes              | “Necesito aprender IA en una semana” |
| Motivación | Te interesa el tema           | “Es lo único que se me ocurre”       |

## Ideas para PI1

### Gestión y organización

- **Gestor de tareas** para un equipo pequeño
- **Control de inventario** para un pequeño negocio
- **Sistema de reservas** para una peluquería, taller o pista deportiva
- **Gestor de gastos** personales o de un viaje
- **Organizador de eventos** (bodas, cumpleaños, conferencias)

### Educación

- **Plataforma de apuntes** compartidos entre estudiantes
- **Gestor de trabajos** para un instituto (entrega, corrección, notas)
- **Sistema de tutorías** (reservar y gestionar sesiones)
- **Generador de horarios** de clase
- **App de Flashcards** para estudiar

### Comunidad

- **Directorio de recursos** de un barrio (talleres, asociaciones)
- **Red de trueque** de objetos entre vecinos
- **Gestor de incidencias** para una comunidad de vecinos
- **App para compartir coche** entre compañeros de clase

### Tiempo real / Datos

- **Tablero Kanban** para equipos pequeños

- **Gestor de hábitos** y seguimiento personal
- **Sistema de encuestas** con resultados en tiempo real
- **Gestor de recetas** con filtros por ingredientes

### *Cómo validar tu idea*

Antes de empezar a programar, valida que tu idea tiene sentido:

1. **Háblala con 3 personas:** ¿les parece útil? ¿la usarían?
2. **Búscala en Google:** ¿ya existe algo así? ¿qué harías diferente?
3. **Pregunta al profesor:** ¿es viable para el tiempo que tenemos?
4. **Recorta alcance:** ¿cuál es la versión más pequeña que sigue siendo útil?

## Ideas para PI2

En PI2 partís del proyecto de PI1. Las ideas se centran en **evolucionar**:

- Añadir autenticación con Google/GitHub
- Integrar pagos simulados (Stripe test)
- Añadir mapas (tiendas cercanas, rutas)
- Implementar chat en tiempo real (WebSockets)
- Dashboard con gráficos y estadísticas
- Exportar datos a PDF/CSV
- Notificaciones por email
- Versión móvil responsive avanzada
- Panel de administración
- Tests automatizados y CI/CD

## Y si no tienes ni idea

Empieza por un problema que **tú mismo tengas**:

- “Necesito organizar mis apuntes”
- “Mi madre necesita gestionar las citas de su negocio”
- “En mi club deportivo no tienen web”
- “Odio hacer la lista de la compra y siempre se me olvida algo”

El mejor proyecto es el que te sirve a ti. Porque si a ti te sirve, lo harás con ganas. Y si lo haces con ganas, sale mejor.